ROB521 Lab 3 Report

Claire Zhang, 1008001822
Trent Rossos, 1008073914
Andreas Arzoumanian,
Nayaab Ali, 1008013917

Table of Contents & List of Figures

Part 1: Vehicle Calibration **2**
Wheel Radius Experiment 2
Wheel Baseline (Separation) Experiment 2
Part 2: Motion Estimation **3**
Figure 1: Plot of Circle Path
Figure 2: Plot of Unique Path **3**
Part 3: Occupancy Grid Mapping **4**
Figure 3: Mapped environment for the closed perimeter of the simulated environment (some edges not fully explored) **4**
Video 5
Part 4: Mapping on a Real Robot **5**
Figure 4: Mapped environment of the real environment with a real robot **5**
Code **6**
l3_estimate_wheel_radius.py 6
l3_estimate_wheel_baseline.py 9
l3_estimate_robot_motion.py 12
l3_mapping.py 12

Part 1: Vehicle Calibration

Wheel Radius Experiment

To determine the wheel radius, the robot was driven forward in a straight line of 75 cm using teleoperation. The code subscribes to the `sensor_state` topic to accumulate encoder ticks on both wheels, and monitors `cmd_vel` to detect when the robot starts and stops. Once stopped, the wheel radius is estimated as the total distance driven (0.75 m) divided by the total angle swept by the wheels, where the wheel angle is computed from encoder ticks using the known ticks-per-revolution constant: `radius = distance / (ticks × 2π / ticks_per_rev)`. The measured wheel radius was 0.033 m, which matches the factory value of 0.033 m from the datasheet exactly. Despite this agreement, one source of uncertainty is surface friction and wheel deformation, if the wheel compresses slightly against the ground, its effective rolling radius is smaller than its physical radius, causing the estimator to underestimate the true radius. This effect could be mitigated by performing the experiment on a flat, hard surface and averaging the result over multiple longer straight-line runs to reduce the relative impact of any single slip or compression event. Additionally, since the robot was operated remotely to driven a predetermined amount of distance, it is impossible to exactly stop at 75cm which results in the calculated radius being slightly off.

Wheel Baseline (Separation) Experiment

To determine the wheel separation, the robot was rotated in place three full rotations (1080°) using teleoperation. Performing three full rotations rather than just one reduces the relative contribution of any single encoder error or slip event to the final estimate. The code tracks the cumulative encoder ticks on each wheel and since the two wheels travel equal and opposite distances during an in-place rotation, the baseline is calculated as the total arc length travelled by one wheel divided by the total rotation angle of the robot body: `baseline = arc_length / (N × π)`, where N is the number of full rotations. The measured baseline was 0.295 m, compared to the factory value of 0.287 m,a discrepancy of approximately 2.8%. A likely source of bias is wheel slip during in-place rotation: because the two wheels push against the ground in opposite directions simultaneously, lateral tire deformation causes the robot to rotate slightly less than the encoders report, making the estimator believe each wheel travelled further per body rotation than it actually did, and therefore overestimating the baseline. This could be mitigated by rotating at a slower speed to reduce slip forces, or by averaging across multiple trials to smooth out inconsistent slip behaviour.

Part 2: Motion Estimation

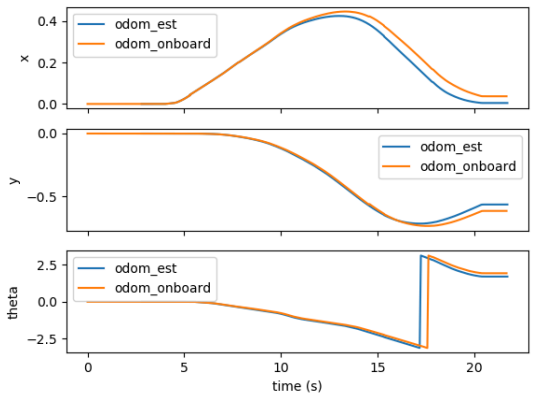

Figure 1: Plot of Circle Path

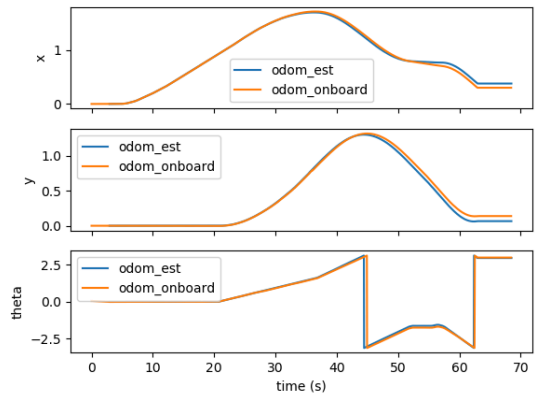

Figure 2: Plot of Unique Path

Figures 1 and 2 compare the onboard and local estimation when moving along a circular path and a “unique” path. In terms of differences in path estimation between local and onboard mapping, the x, y and theta plots are quite similar. During circular path and unique path following, there is minor deviation at 17s and 45s, respectively. A possible source of error that accounts for the differences could be imperfections in the motors/encoders at high speed directional changes. The onboard estimation and local “perfect” estimation is not identical despite tuning of parameters in part 1. In terms of differences in path estimation between physical movement and onboard mapping, we can confirm the same source of error that accounts for the differences at high speed directional changes. Directional, meaning at curving quickly in the circular path and zigzag-ing in the “unique” path.

Part 3: Occupancy Grid Mapping

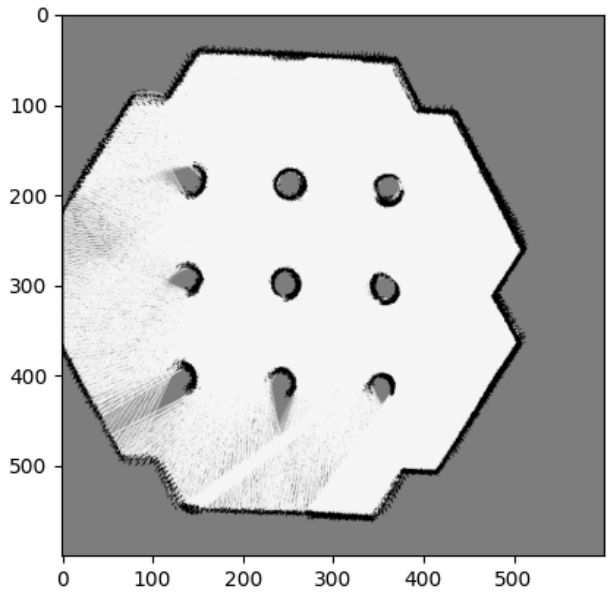

Figure 3: Mapped environment for the closed perimeter of the simulated environment (some edges not fully explored)

The code for l3_mapping.py updates the occupancy grid's log likelihood for each LiDAR measurement. As new LiDAR data arrives, the scan_cb function is called to iterate through each range measurement in the scan message. It computes the angle of each ray by combining the robot heading with the ray angle offset. For each valid measurement, ray_trace_update is called. The ray_trace_update function updates the occupancy grid along one ray. Cells between the robot and the measured obstacle are free, and thus their log-odds are decreased by BETA. The cells at the endpoint of the ray are where the obstacle was detected, so their log-odds are increased by ALPHA. ALPHA is set to 3 and BETA is set to 1, which makes the algorithm consider observations of obstacles more heavily weighted. The log_odds are also capped such that a free space cell caps at -3.0, and an occupied cell caps at 10.0, making it so that no single region of the map can become overwhelmingly certain in either direction, allowing the map to still be updated if new contradicting evidence arrives. These log-odds are then converted to probabilities and stored as integers between 0 and 100 in the map.

One potential source of error in the algorithm is from the robot's dead-reckoning with odometry drift. As time passes, small errors in the odometry accumulate which causes the estimated pose of the robot to diverge from ground-truth. As such, obstacles detected at different times get projected onto inconsistent grid locations and may cause the mapped environment to blur or shift.

Video
A video of the rviz map for the simulated environment mapping was obtained. It starts with a map that has not been updated with any LiDAR measurements. Google Drive viewing link here.

Part 4: Mapping on a Real Robot

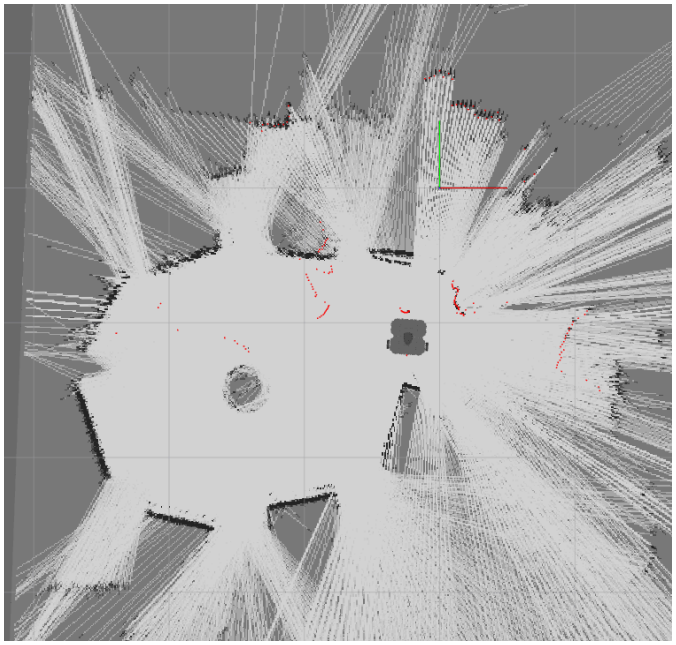

Figure 4: Mapped environment of the real environment with a real robot.

The map captures the in-lab maze but shows noise and distortion, including blurred walls and misaligned features, likely due to pose estimation error caused by wheel slippage, uneven ground, and other real-world effects not present in simulation, leading to drift in the robot's estimated position.

Code

l3_estimate_wheel_radius.py

```
#!/usr/bin/env python3

import rospy
import numpy as np
import threading
from turtlebot3_msgs.msg import SensorState
from std_msgs.msg import Empty
from geometry_msgs.msg import Twist

INT32_MAX = 2**31
DRIVEN_DISTANCE = 0.75 #in meters
TICKS_PER_ROTATION = 4096

class wheelRadiusEstimator():
    def __init__(self):
        rospy.init_node('encoder_data', anonymous=True) # Initialize node

        #Subscriber bank
        rospy.Subscriber("cmd_vel", Twist, self.startStopCallback)
        rospy.Subscriber("sensor_state", SensorState, self.sensorCallback)
#Subscribe to the sensor state msg

        #Publisher bank
        self.reset_pub = rospy.Publisher('reset', Empty, queue_size=1)

        #Initialize variables
        self.left_encoder_prev = None
        self.right_encoder_prev = None
        self.del_left_encoder = 0
        self.del_right_encoder = 0
        self.isMoving = False #Moving or not moving
        self.lock = threading.Lock()

        #Reset the robot
        reset_msg = Empty()
```

```
        self.reset_pub.publish(reset_msg)
        print('Ready to start wheel radius calibration!')
        return

    def safeDelPhi(self, a, b):
        #Need to check if the encoder storage variable has overflowed
        diff = np.int64(b) - np.int64(a)
        if diff < -np.int64(INT32_MAX): #Overflowed
            delPhi = (INT32_MAX - 1 - a) + (INT32_MAX + b) + 1
        elif diff > np.int64(INT32_MAX) - 1: #Underflowed
            delPhi = (INT32_MAX + a) + (INT32_MAX - 1 - b) + 1
        else:
            delPhi = b - a
        return delPhi

    def sensorCallback(self, msg):
        #Retrieve the encoder data form the sensor state msg
        self.lock.acquire()
        if self.left_encoder_prev is None or self.left_encoder_prev is
None:
            self.left_encoder_prev = msg.left_encoder #int32
            self.right_encoder_prev = msg.right_encoder #int32
        else:
            #Calculate and integrate the change in encoder value
            self.del_left_encoder +=
self.safeDelPhi(self.left_encoder_prev, msg.left_encoder)
            self.del_right_encoder +=
self.safeDelPhi(self.right_encoder_prev, msg.right_encoder)

            #Store the new encoder values
            self.left_encoder_prev = msg.left_encoder #int32
            self.right_encoder_prev = msg.right_encoder #int32
        self.lock.release()
        return

    def startStopCallback(self, msg):
        input_velocity_mag = np.linalg.norm(np.array([msg.linear.x,
msg.linear.y, msg.linear.z]))
        if self.isMoving is False and np.absolute(input_velocity_mag) > 0:
            self.isMoving = True #Set state to moving
```

```
            print('Starting Calibration Procedure')

        elif self.isMoving is True and np.isclose(input_velocity_mag, 0):
            self.isMoving = False #Set the state to stopped

            # # YOUR CODE HERE!!!-------------------------------------
            # Calculate the radius of the wheel based on encoder
measurements

            # radius = ##
            # print('Calibrated Radius: {} m'.format(radius))

            # Convert encoder ticks to wheel rotation
            delta_phi_l = (2*np.pi * self.del_left_encoder) /
TICKS_PER_ROTATION
            delta_phi_r = (2*np.pi * self.del_right_encoder) /
TICKS_PER_ROTATION

            # Average rotation
            delta_phi_avg = (delta_phi_l + delta_phi_r) / 2

            # Wheel radius estimate
            radius = DRIVEN_DISTANCE / delta_phi_avg

            print('Calibrated Radius: {} m'.format(radius))

            #-------------------------------------

            #Reset the robot and calibration routine
            self.lock.acquire()
            self.left_encoder_prev = None
            self.right_encoder_prev = None
            self.del_left_encoder = 0
            self.del_right_encoder = 0
            self.lock.release()
            reset_msg = Empty()
            self.reset_pub.publish(reset_msg)
            print('Resetted the robot to calibrate again!')

        return
```

```
if __name__ == '__main__':
    Estimator = wheelRadiusEstimator() #create instance
    rospy.spin()
```

l3_estimate_wheel_baseline.py

```
#!/usr/bin/env python3

import rospy
import numpy as np
import threading
from turtlebot3_msgs.msg import SensorState
from std_msgs.msg import Empty
from geometry_msgs.msg import Twist

INT32_MAX = 2**31
NUM_ROTATIONS = 3
TICKS_PER_ROTATION = 4096
WHEEL_RADIUS = 0.066 / 2 #In meters


class wheelBaselineEstimator():
    def __init__(self):
        rospy.init_node('encoder_data', anonymous=True) # Initialize node

        #Subscriber bank
        rospy.Subscriber("cmd_vel", Twist, self.startStopCallback)
        rospy.Subscriber("sensor_state", SensorState, self.sensorCallback)
#Subscribe to the sensor state msg

        #Publisher bank
        self.reset_pub = rospy.Publisher('reset', Empty, queue_size=1)

        #Initialize variables
        self.left_encoder_prev = None
        self.right_encoder_prev = None
```

```
        self.del_left_encoder = 0
        self.del_right_encoder = 0
        self.isMoving = False #Moving or not moving
        self.lock = threading.Lock()

        #Reset the robot
        reset_msg = Empty()
        self.reset_pub.publish(reset_msg)
        print('Ready to start wheel radius calibration!')
        return

    def safeDelPhi(self, a, b):
        #Need to check if the encoder storage variable has overflowed
        diff = np.int64(b) - np.int64(a)
        if diff < -np.int64(INT32_MAX): #Overflowed
            delPhi = (INT32_MAX - 1 - a) + (INT32_MAX + b) + 1
        elif diff > np.int64(INT32_MAX) - 1: #Underflowed
            delPhi = (INT32_MAX + a) + (INT32_MAX - 1 - b) + 1
        else:
            delPhi = b - a
        return delPhi

    def sensorCallback(self, msg):
        #Retrieve the encoder data form the sensor state msg
        self.lock.acquire()
        if self.left_encoder_prev is None or self.left_encoder_prev is
None:
            self.left_encoder_prev = msg.left_encoder #int32
            self.right_encoder_prev = msg.right_encoder #int32
        else:
            #Calculate and integrate the change in encoder value
            self.del_left_encoder +=
self.safeDelPhi(self.left_encoder_prev, msg.left_encoder)
            self.del_right_encoder +=
self.safeDelPhi(self.right_encoder_prev, msg.right_encoder)

            #Store the new encoder values
            self.left_encoder_prev = msg.left_encoder #int32
            self.right_encoder_prev = msg.right_encoder #int32
        self.lock.release()
```

```
        return

    def startStopCallback(self, msg):
        if self.isMoving is False and np.absolute(msg.angular.z) > 0:
            self.isMoving = True #Set state to moving
            print('Starting Calibration Procedure')

        elif self.isMoving is True and np.isclose(msg.angular.z, 0):
            self.isMoving = False #Set the state to stopped

            # # YOUR CODE HERE!!!----------------------------------
            # Calculate the radius of the wheel based on encoder
measurements

            # separation = ##
            # print('Calibrated Separation: {} m'.format(separation))

            # Wheel arc lengths
            s_l = WHEEL_RADIUS * 2 * np.pi * (self.del_left_encoder /
TICKS_PER_ROTATION)
            s_r = WHEEL_RADIUS * 2 * np.pi * (self.del_right_encoder /
TICKS_PER_ROTATION)

            # Total robot rotation
            delta_theta = 2 * np.pi * NUM_ROTATIONS

            # Baseline estimate
            separation = (s_l - s_r) / delta_theta

            print('Calibrated Separation: {} m'.format(separation))
            #--------------------------------

            #Reset the robot and calibration routine
            self.lock.acquire()
            self.left_encoder_prev = None
            self.right_encoder_prev = None
            self.del_left_encoder = 0
            self.del_right_encoder = 0
            self.lock.release()
            reset_msg = Empty()
```

```
            self.reset_pub.publish(reset_msg)
            print('Resetted the robot to calibrate again!')

        return


if __name__ == '__main__':
    Estimator = wheelBaselineEstimator() #create instance
    rospy.spin()
```

l3_estimate_robot_motion.py

```
#!/usr/bin/env python3
from __future__ import division, print_function
import time

import numpy as np
import rospy
import tf_conversions
import tf2_ros
import rosbag
import rospkg

# msgs
from turtlebot3_msgs.msg import SensorState
from nav_msgs.msg import Odometry
from geometry_msgs.msg import Pose, Twist, TransformStamped, Transform,
Quaternion
from std_msgs.msg import Empty

from utils import convert_pose_to_tf, euler_from_ros_quat,
ros_quat_from_euler


ENC_TICKS = 4096
RAD_PER_TICK = 0.001533981
WHEEL_RADIUS = .066 / 2
BASELINE = .287 / 2
```

```
class WheelOdom:
    def __init__(self):
        # publishers, subscribers, tf broadcaster
        self.sensor_state_sub = rospy.Subscriber('/sensor_state',
SensorState, self.sensor_state_cb, queue_size=1)
        self.odom_sub = rospy.Subscriber('/odom', Odometry, self.odom_cb,
queue_size=1)
        self.wheel_odom_pub = rospy.Publisher('/wheel_odom', Odometry,
queue_size=1)
        self.tf_br = tf2_ros.TransformBroadcaster()

        # attributes
        self.odom = Odometry()
        self.odom.pose.pose.position.x = 1e10
        self.wheel_odom = Odometry()
        self.wheel_odom.header.frame_id = 'odom'
        self.wheel_odom.child_frame_id = 'wo_base_link'
        self.wheel_odom_tf = TransformStamped()
        self.wheel_odom_tf.header.frame_id = 'odom'
        self.wheel_odom_tf.child_frame_id = 'wo_base_link'
        self.pose = Pose()
        self.pose.orientation.w = 1.0
        self.twist = Twist()
        self.last_enc_l = None
        self.last_enc_r = None
        self.last_time = None

        # rosbag
        rospack = rospkg.RosPack()
        path = rospack.get_path("rob521_lab3")
        self.bag = rosbag.Bag(path+"/motion_estimate.bag", 'w')

        # reset current odometry to allow comparison with this node
        reset_pub = rospy.Publisher('/reset', Empty, queue_size=1,
latch=True)
        reset_pub.publish(Empty())
        while not rospy.is_shutdown() and (self.odom.pose.pose.position.x
>= 1e-3 or self.odom.pose.pose.position.y >= 1e-3 or
```

```
                self.odom.pose.pose.orientation.z >= 1e-2):
            time.sleep(0.2)  # allow reset_pub to be ready to publish
        print('Robot odometry reset.')

        rospy.spin()
        self.bag.close()
        print("saving bag")

    def sensor_state_cb(self, sensor_state_msg):
        # Callback for whenever a new encoder message is published
        # set initial encoder pose
        if self.last_enc_l is None:
            self.last_enc_l = sensor_state_msg.left_encoder
            self.last_enc_r = sensor_state_msg.right_encoder
            self.last_time = sensor_state_msg.header.stamp
        else:
            # update calculated pose and twist with new data
            le = sensor_state_msg.left_encoder
            re = sensor_state_msg.right_encoder

            # # YOUR CODE HERE!!!-----------------------------
            # Update your odom estimates with the latest encoder
measurements and populate the relevant area
            # of self.pose and self.twist with estimated position, heading
and velocity

            # self.pose.position.x = xx
            # self.pose.position.y = xx
            # self.pose.orientation = xx

            # self.twist.linear.x = mu_dot[0].item()
            # self.twist.linear.y = mu_dot[1].item()
            # self.twist.angular.z = mu_dot[2].item()

            current_time = sensor_state_msg.header.stamp
            dt = (current_time - self.last_time).to_sec()

            delta_le = (le - self.last_enc_l) * RAD_PER_TICK
            delta_re = (re - self.last_enc_r) * RAD_PER_TICK
```

```
        # Distance for each wheel
        dL = delta_le * WHEEL_RADIUS
        dR = delta_re * WHEEL_RADIUS

        # Avg dist travelled
        d_centre = (dL + dR) / 2

        delta_theta = (dR - dL) / (BASELINE * 2)
        theta_prev = euler_from_ros_quat(self.pose.orientation)[2]
        theta = theta_prev + delta_theta

        dx = d_centre * np.cos(theta_prev + delta_theta / 2)
        dy = d_centre * np.sin(theta_prev + delta_theta / 2)

        # Updates
        self.pose.position.x += dx
        self.pose.position.y += dy
        self.pose.orientation = ros_quat_from_euler((0, 0, theta))

        self.twist.linear.x = d_centre / dt
        self.twist.linear.y = 0
        self.twist.angular.z = delta_theta / dt

        self.last_enc_l = le
        self.last_enc_r = re
        self.last_time = current_time

        #---------------------------------------------

        # publish the updates as a topic and in the tf tree
        current_time = rospy.Time.now()
        self.wheel_odom_tf.header.stamp = current_time
        self.wheel_odom_tf.transform = convert_pose_to_tf(self.pose)
        self.tf_br.sendTransform(self.wheel_odom_tf)

        self.wheel_odom.header.stamp = current_time
        self.wheel_odom.pose.pose = self.pose
        self.wheel_odom.twist.twist = self.twist
        self.wheel_odom_pub.publish(self.wheel_odom)
```

```
            self.bag.write('odom_est', self.wheel_odom)

            #for testing against actual odom
            print("Wheel Odom: x: %2.3f, y: %2.3f, t: %2.3f" % (
                self.pose.position.x, self.pose.position.y, theta
            ))
            print("Turtlebot3 Odom: x: %2.3f, y: %2.3f, t: %2.3f" % (
                self.odom.pose.pose.position.x,
self.odom.pose.pose.position.y,
                euler_from_ros_quat(self.odom.pose.pose.orientation)[2]
            ))

    def odom_cb(self, odom_msg):
        # get odom from turtlebot3 packages
        self.odom = odom_msg
        self.bag.write('odom_onboard', self.odom)

    def plot(self, bag):
        data = {"odom_est":{"time":[], "data":[]},
                "odom_onboard":{'time':[], "data":[]}}
        for topic, msg, t in bag.read_messages(topics=['odom_est',
'odom_onboard']):
            print(msg)


if __name__ == '__main__':
    try:
        rospy.init_node('wheel_odometry')
        wheel_odom = WheelOdom()
    except rospy.ROSInterruptException:
        pass
```

l3_mapping.py

```
#!/usr/bin/env python3
from __future__ import division, print_function

import numpy as np
import rospy
```

```
import tf2_ros
from skimage.draw import line as ray_trace
import rospkg
import matplotlib.pyplot as plt

# msgs
from nav_msgs.msg import OccupancyGrid, MapMetaData
from geometry_msgs.msg import TransformStamped
from sensor_msgs.msg import LaserScan

from utils import convert_pose_to_tf, convert_tf_to_pose,
euler_from_ros_quat, \
     tf_to_tf_mat, tf_mat_to_tf


ALPHA = 3
BETA = 1
MAP_DIM = (6, 6)
CELL_SIZE = .01
NUM_PTS_OBSTACLE = 3
SCAN_DOWNSAMPLE = 1

class OccupancyGripMap:
    def __init__(self):
        # use tf2 buffer to access transforms between existing frames in
tf tree
        self.tf_buffer = tf2_ros.Buffer()
        self.listener = tf2_ros.TransformListener(self.tf_buffer)
        self.tf_br = tf2_ros.TransformBroadcaster()

        # subscribers and publishers
        self.scan_sub = rospy.Subscriber('/scan', LaserScan, self.scan_cb,
queue_size=1)
        self.map_pub = rospy.Publisher('/map', OccupancyGrid,
queue_size=1)

        # attributes
        width = int(MAP_DIM[0] / CELL_SIZE); height = int(MAP_DIM[1] /
CELL_SIZE)
        self.log_odds = np.zeros((width, height))
```

```
        self.np_map = np.ones((width, height), dtype=np.uint8) * -1  # -1
for unknown
        self.map_msg = OccupancyGrid()
        self.map_msg.info = MapMetaData()
        self.map_msg.info.resolution = CELL_SIZE
        self.map_msg.info.width = width
        self.map_msg.info.height = height

        # transforms
        self.base_link_scan_tf =
self.tf_buffer.lookup_transform('base_link', 'base_scan', rospy.Time(0),

rospy.Duration(2.0))
        odom_tf = self.tf_buffer.lookup_transform('odom', 'base_link',
rospy.Time(0), rospy.Duration(2.0)).transform

        # set origin to center of map
        rob_to_mid_origin_tf_mat = np.eye(4)
        rob_to_mid_origin_tf_mat[0, 3] = -width / 2 * CELL_SIZE
        rob_to_mid_origin_tf_mat[1, 3] = -height / 2 * CELL_SIZE
        odom_tf_mat = tf_to_tf_mat(odom_tf)
        self.map_msg.info.origin =
convert_tf_to_pose(tf_mat_to_tf(odom_tf_mat.dot(rob_to_mid_origin_tf_mat))
)

        # map to odom broadcaster
        self.map_odom_timer = rospy.Timer(rospy.Duration(0.1),
self.broadcast_map_odom)
        self.map_odom_tf = TransformStamped()
        self.map_odom_tf.header.frame_id = 'map'
        self.map_odom_tf.child_frame_id = 'odom'
        self.map_odom_tf.transform.rotation.w = 1.0

        rospy.spin()
        plt.imshow(100-self.np_map, cmap='gray', vmin=0, vmax=100)
        rospack = rospkg.RosPack()
        path = rospack.get_path("rob521_lab3")
        plt.savefig(path+"/map.png")

    def broadcast_map_odom(self, e):
```

```
        self.map_odom_tf.header.stamp = rospy.Time.now()
        self.tf_br.sendTransform(self.map_odom_tf)

    def scan_cb(self, scan_msg):
        # read new laser data and populate map
        # get current odometry robot pose
        try:
            odom_tf = self.tf_buffer.lookup_transform('odom', 'base_scan',
rospy.Time(0)).transform
        except tf2_ros.TransformException:
            rospy.logwarn('Pose from odom lookup failed. Using origin as
odom.')
            odom_tf = convert_pose_to_tf(self.map_msg.info.origin)

        # get odom in frame of map
        odom_map_tf = tf_mat_to_tf(

np.linalg.inv(tf_to_tf_mat(convert_pose_to_tf(self.map_msg.info.origin))).
dot(tf_to_tf_mat(odom_tf))
        )
        odom_map = np.zeros(3)
        odom_map[0] = odom_map_tf.translation.x
        odom_map[1] = odom_map_tf.translation.y
        odom_map[2] = euler_from_ros_quat(odom_map_tf.rotation)[2]

        # YOUR CODE HERE!!! -------------------------------------------
        #Loop through each measurement in scan_msg to get the correct
angle and
        # x_start and y_start to send to your ray_trace_update function.

        x_start = int(odom_map[0] / CELL_SIZE)
        y_start = int(odom_map[1] / CELL_SIZE)

        for i in range(0, len(scan_msg.ranges), SCAN_DOWNSAMPLE):
            range_mes = scan_msg.ranges[i]
            if np.isnan(range_mes) or np.isinf(range_mes):
                continue
            # Angle of this ray in map frame
            angle = odom_map[2] + scan_msg.angle_min + i *
scan_msg.angle_increment
```

```
            self.np_map, self.log_odds = self.ray_trace_update(
                self.np_map, self.log_odds, x_start, y_start, angle,
range_mes
            )

        #-------------------------------------
        # publish the message
        self.map_msg.info.map_load_time = rospy.Time.now()
        self.map_msg.data = self.np_map.flatten()
        self.map_pub.publish(self.map_msg)

    def ray_trace_update(self, map, log_odds, x_start, y_start, angle,
range_mes):
        """
        A ray tracing grid update as described in the lab document.

        :param map: The numpy map.
        :param log_odds: The map of log odds values.
        :param x_start: The x starting point in the map coordinate frame
(i.e. the x 'pixel' that the robot is in).
        :param y_start: The y starting point in the map coordinate frame
(i.e. the y 'pixel' that the robot is in).
        :param angle: The ray angle relative to the x axis of the map.
        :param range_mes: The range of the measurement along the ray.
        :return: The numpy map and the log odds updated along a single
ray.
        """
        # YOUR CODE HERE!!! ----------------------------------------
        #You should modify the log_odds object and the numpy map based on
the outputs from
        # ray_trace and the equations from class. Your numpy map must be
an array of int8s with 0 to 100 representing
        # probability of occupancy, and -1 representing unknown.
        # Compute endpoint of ray in pixel coordinates


        # Compute endpoint of ray in pixel coordinates
        x_end = int(x_start + (range_mes / CELL_SIZE) * np.cos(angle))
        y_end = int(y_start + (range_mes / CELL_SIZE) * np.sin(angle))
```

```
        # Get all pixels along the ray
        width, height = log_odds.shape
        x_end_clamped = np.clip(x_end, 0, width - 1)
        y_end_clamped = np.clip(y_end, 0, height - 1)
        x_start_clamped = np.clip(x_start, 0, width - 1)
        y_start_clamped = np.clip(y_start, 0, height - 1)

        rr, cc = ray_trace(y_start_clamped, x_start_clamped,
y_end_clamped, x_end_clamped)

        # Free cells: all pixels along ray except last NUM_PTS_OBSTACLE
        free_end = max(0, len(rr) - NUM_PTS_OBSTACLE)
        if free_end > 0:
            log_odds[rr[:free_end], cc[:free_end]] -= BETA

        # Occupied cells: last NUM_PTS_OBSTACLE pixels (only if ray
endpoint is in bounds)
        if 0 <= x_end < width and 0 <= y_end < height:
            log_odds[rr[free_end:], cc[free_end:]] += ALPHA

        np.clip(log_odds, -3.0, 10.0, out=log_odds)

        # Convert log odds to probability, then to 0-100 int8
        probs = self.log_odds_to_probability(log_odds)
        map = (probs * 100).astype(np.int8)

        #-----------------------------------------------------

        return map, log_odds

    def log_odds_to_probability(self, values):
        # print(values)
        return np.exp(values) / (1 + np.exp(values))


if __name__ == '__main__':
    try:
        rospy.init_node('mapping')
        ogm = OccupancyGripMap()
    except rospy.ROSInterruptException:
```

```
        pass
```